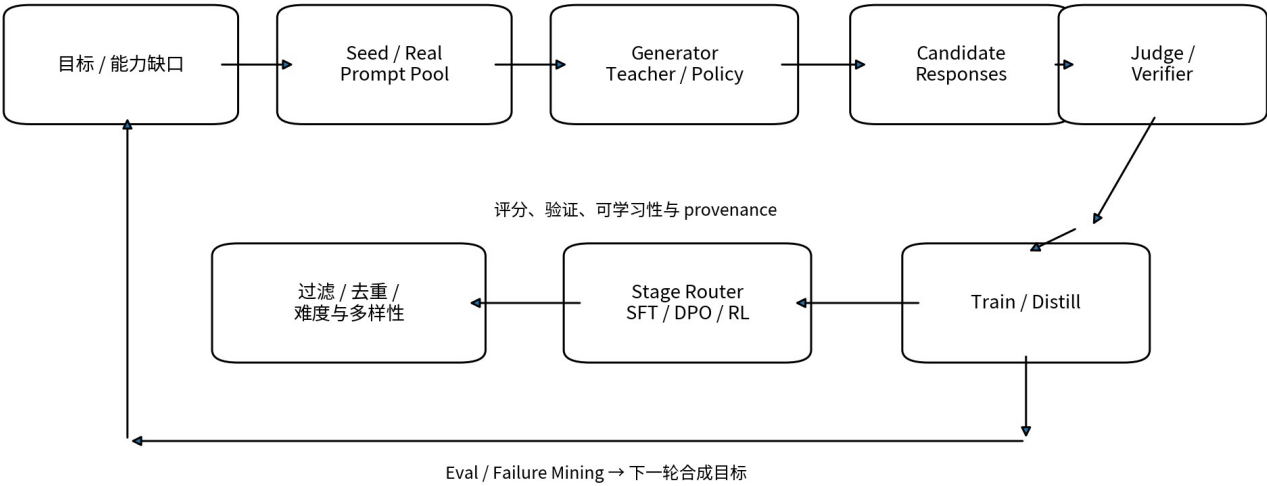


LLM 后训练之 Synthetic Data

合成数据：从“低成本扩量”到“可控经验生成系统”

Synthetic Data Factory：从“生成文本”到“制造可学习经验”



Deep Research / Engineering Handbook

版本：2026-08-13

究范：SFT / Preference / RLVR / Reasoning / Agent / Distillation

执行摘要：Synthetic Data 真正解决什么？

在现代 LLM 后训练中，Synthetic Data 已经不是“缺数据时让大模型帮忙写一些样本”的附属技巧，而逐渐成为一个可编程的 Experience Factory。它负责把能力缺口、真实用户分布、教师模型能力、验证器与训练阶段编译成可学习的 Prompt、Response、Reasoning、Preference、Critique 和 Agent Trajectory。本手册的核心观点是：合成数据的价值不由“是否由模型生成”决定，而由它是否提供当前 policy 尚未掌握、且可验证、可归因、分布可控的训练信号决定。

工程推论 | 核心公式

$$\text{Effective Synthetic Experience} \approx (\text{Coverage} \times \text{Correctness} \times \text{Learnability} \times \text{Diversity} \times \text{Policy-Relevance} \times \text{Verifiability}) / (\text{Generation Cost} + \text{Verification Cost} + \text{Collapse Risk})$$

结论	Know-Why	直接工程含义
1. Synthetic ≠ Generated Text	训练对象可能是 prompt、偏好、reward label、reasoning、trajectory，而不仅是 answer。	先定义 supervision unit，再设计 generator。
2. Teacher 强不等于数据强	强模型也会产生偏见、同质化、不可验证 reasoning。	Teacher、Generator Policy、Verifier 必须解耦。
3. 先问“可验证吗？”	合成数据最大优势是可批量生成；最大风险是错误也被批量复制。	数学/代码优先接入 deterministic verifier；开放任务使用 judge ensemble + audit。
4. 真实数据仍是锚点	递归训练在理论和实验中可能丢失分布尾部。	保留 real-data anchor、真实用户分布与 provenance。
5. Stage-specific	SFT、DPO、RLVR、Distill 对“好数据”的定义不同。	同一 synthetic pool 必须经 Stage Router，而不是全阶段复用。
6. 动态比静态更重要	模型能力变化后，旧 synthetic data 会快速变 easy/stale。	Eval → Gap → Generate → Verify → Train → Eval 的闭环优于一次性大数据集。

目录

- 01 第一性原理：Synthetic Data 的边界与目标
- 02 Taxonomy：七类合成训练信号
- 03 历史演进：Self-Instruct → OpenThoughts
- 04 Prompt Synthesis
- 05 Response / Demonstration Synthesis
- 06 Reasoning Trace Synthesis
- 07 Preference / Feedback Synthesis
- 08 Critique、Correction 与 Negative Data
- 09 Agent / Tool Trajectory Synthesis
- 10 Generator / Teacher 设计
- 11 Verifier 与过滤
- 12 Diversity、Difficulty 与 Learnable Frontier
- 13 Synthetic Data Mixing 与 Reality Anchor
- 14 Model Collapse / Synthetic Monoculture
- 15 Stage Router：SFT / DPO / RLVR / Distill

- 16 Dynamic Data Flywheel
- 17 成本、吞吐与基础设施
- 18 数据契约、Provenance 与污染防火墙
- 19 生产级 Recipes
- 20 Failure Modes
- 21 成熟度模型
- 22 论文地图与阅读路线

01 | 第一性原理：Synthetic Data 的边界与目标

一个容易误导的定义是：“Synthetic Data = LLM 生成的数据”。这只描述来源，不描述训练价值。后训练真正关心的是：某个生成过程是否创造了新的、可靠的监督信号。

对象	Natural / Human	Synthetic	Hybrid
Prompt	真实用户请求、人工题目	LLM 自生成、persona 生成、evolution	真实 seed + rewrite/evolution
Response	人工示范	Teacher completion / rejection sampling	Teacher 生成 + 人工审计
Reasoning	人工 CoT	Teacher reasoning / self-rationale	答案由 verifier, reasoning 由 teacher
Preference	人工 pairwise label	LLM Judge / Rule-derived pair	AI 预标 + human disagreement review
Trajectory	真实操作日志	模拟环境/teacher agent rollout	真实环境 + synthetic reflection/correction

研究事实 | Self-Instruct
Self-Instruct 用模型生成 instruction、input 与 output，过滤无效/相似样本后回训；论文报告在 Super-NaturalInstructions 上相对原始 GPT-3 获得 33 个百分点的绝对提升。[1]

工程推论 | 工程边界
不要把 “synthetic / real” 当成质量标签。更有用的元数据是：source、generator、generation policy、verification method、lineage、difficulty、novelty、policy distance。

Synthetic Data Taxonomy：合成的不是一种“数据”，而是不同训练信号

	主要目的	典型监督信号	关键风险
Prompt	扩大任务覆盖	任务本身	同质化 / 无效任务
Response	SFT / 蒸馏	理想回答	教师错误 / 风格复制
Reasoning	推理能力迁移	CoT / 步骤	伪推理 / 答案泄漏
Preference	DPO / RM	chosen > rejected	Judge bias / 低信息 pair
Critique / Correction	自纠错 / 过程监督	错误 → 诊断 → 修正	把错误步骤学进去
Tool / Agent Trajectory	Agent SFT / RL	状态-动作-观察	环境偏差 / 长程误差传播
Reward / Verifier Label	RLVR / 筛选	score / label	reward hacking / 代理目标

图 1 | Synthetic Data 的 supervision-unit taxonomy

02 | Taxonomy：七类合成训练信号

类型	输入	输出	训练阶段	核心问题
Prompt Synthesis	seed / persona / corpus	task / query	所有阶段	有没有覆盖当前能力缺口？
Response Synthesis	prompt + teacher	ideal response	SFT / distill	答案是否正确且可模仿？
Reasoning Synthesis	problem + teacher	reasoning trace	Reasoning SFT	过程是真实有用还是伪解释？
Preference Synthesis	prompt + candidates + judge	chosen/rejected	DPO / RM	pair 是否有信息量且 judge 可靠？
Critique / Correction	bad trajectory + critic	diagnosis + corrected response	SFT / PRM	错误步骤是否被正确 mask？
Agent Trajectory	task + env + agent	state/action/obs trace	Agent SFT/RL	环境反馈能否验证最终成功？
Reward / Label Synthesis	candidate + verifier	score / label	RLVR / filter	代理奖励是否被 gaming？

03 | 历史演进：从 Bootstrapping 到可验证经验工厂

阶段	代表工作	关键突破	尚未解决
2022 Bootstrapping	Self-Instruct [1] / Unnatural Instructions [2]	模型自造 instruction 与答案，降低人工成本	噪声、同质化、缺少复杂度控制
2022 Reasoning self-training	STaR [3]	仅保留能导向正确答案的 rationale，迭代自举	依赖可判定 final answer；错误 rationale 过滤难
2023 Complexity evolution	WizardLM / Evol-Instruct [4]	通过 rewrite/constraint 深化 instruction	复杂 ≠ 有价值；可能只变长
2023 Teacher explanation	Orca [5]	学习强教师的 explanation traces，而非浅层输出	教师风格复制与 reasoning faithfulness
2023 AI Feedback	UltraFeedback [6]	规模化 AI preference feedback	Judge bias / preference proxy
2024 Template elicitation	Magpie [7]	几乎不需要 seed，利用 chat template 诱出 user queries	分布受 generator 模型先验影响
2024 Persona scaling	Persona Hub [8]	persona 作为 diversity control variable	persona 多样 ≠ task 多样；真实性问题
2024 Industrial alignment	Nemotron-4 [9]	对齐数据中 >98% 为合成，并公开生成 pipeline	强依赖生成/奖励模型与高算力
2024 Open recipe	Tülu 3 [10]	real + synthetic prompts；on/off-policy synthetic preferences	混合与 stage routing 仍需大量 ablation
2025 Reasoning factory	DeepSeek-R1 [11] / OpenThoughts [12] / s1 [13]	长推理轨迹 + verifier + 数据 recipe 系统实验	过长 CoT、teacher artifacts、成本
2025-26 Fully synthetic / privacy	SAO [17] / DPPrefSyn [18]	prompt/response/preference 全自合成；隐私保护合成偏好	闭环自评估偏差与真实分布锚定

Magpie 从对齐后的 Llama-3-Instruct 的 chat template 中“抽取”用户 query，生成约 400 万 instruction，并筛到 30 万高质量样本，展示了从“seed-driven”向“model-prior elicitation”的变化。[7]

研究事实 | Nemotron-4

Nemotron-4 340B 报告其 alignment data 中超过 98% 为合成数据，并将模型定位为 synthetic-data generator；这说明在工业级 recipe 中，Synthetic Data 已可以成为主数据源，但前提是存在完整的生成、奖励与过滤基础设施。[9]

04 | Prompt Synthesis：合成“问题”比合成“答案”更先决

Prompt 决定模型将经历什么任务分布。Prompt Synthesis 的核心不是换句话说，而是扩展 task manifold：能力维度、领域、难度、交互形式、约束、语言、上下文长度与风险边界。

方法	Mechanism	优点	风险 / 适用
Seed Bootstrapping	few human seeds → model generates more tasks	简单、低成本	容易围绕 seed 局部扩散
Paraphrase / Rewrite	同义改写 / 视角改写	提高语言表面多样性	不一定提高技能覆盖
Evol-Instruct	添加约束、深化、具体化、增加 reasoning step	主动制造复杂任务	复杂度可伪增长
Persona-conditioned	persona → user query	增加知识/身份视角	persona 先验偏置；需 task diversity check
Template Elicitation	只给 chat template 左半部分，让 aligned model 补 user turn	减少 seed 依赖	生成分布受模型内部 user prior 限制
Source-grounded	文档/代码/数据库 → 生成问题	可追溯、可验证	受 source coverage 限制
Failure-driven	Eval failure / log → 变体、对抗、边界 prompt	最贴当前 policy	容易 overfit failure cluster

研究事实 | Persona Hub

Persona Hub 提出从网页自动整理 10 亿 persona，并展示其在数学/逻辑题、instruction、工具等合成任务上的用途。[8]

工程推论 | 工程推论：Prompt Coverage Map

对 prompt 做 embedding 去重只解决“文本相似”。生产系统更应该对 capability tag、difficulty、source、constraint type、tool requirement、expected verifier 进行分层覆盖分析。

05 | Response / Demonstration Synthesis：Teacher 不是答案数据库

策略	生成方式	什么时候有价值	主要风险
Single Teacher	一个强模型直接生成	低成本、任务清晰	系统性错误被复制
Teacher Ensemble	多模型候选	可提高 diversity / robustness	成本高；模型相关性导致伪 ensemble
Best-of-N	同 teacher 多采样 + judge/verifier 选优	可验证任务、高价值 SFT	winner style bias；只保留“容易验证”模式
Rejection Sampling	不满足规则/答案则丢弃重采	数学/代码/格式任务	难题样本被过度丢弃
Progressive Teacher	easy teacher trace → strong teacher trace / curriculum	能力迁移	pipeline 复杂
Self-Distill	同模型生成再回训	成本低、风格一致	confirmation bias / diversity collapse

研究事实 | Orca

Orca 的核心不是简单 imitation，而是利用 GPT-4 的 explanation traces 和 progressive learning，强调 richer teacher signal 对小模型能力迁移的重要性。[5]

工程推论 | 关键指标

Teacher Pass Rate、Verifier Pass Rate、Unique Skill Coverage、Pairwise Win Rate、Length Distribution、Judge Disagreement、Cost / Accepted Sample。

06 | Reasoning Trace Synthesis：合成 CoT 的真正难点

Reasoning 数据的监督价值通常高于简单 final answer，但风险也更大：一个最终答案正确的轨迹并不保证所有中间步骤都正确；一个更长的轨迹也不代表更强的 reasoning。

模式	机制	优点	关键风险
Answer-filtered CoT	生成多个 rationale，仅保留 final answer 正确者	STaR 风格、简单有效	可能存在 lucky guess / invalid intermediate
Teacher Long-CoT Distill	强 reasoning model → long trace	可迁移搜索模式与自检	teacher artifacts / 语言风格/过长
Process-verified	step checker / PRM 逐步验证	更强 credit assignment	verifier 构建成本高
Outcome + Trace Heuristic	final verifier + format/length/repetition checks	工程易落地	仍无法保证过程 faithful
Negative/Repair Trace	错误 → critique → corrected trace	学习恢复能力	错误 token 如果 mask 不当会被模仿

研究事实 | STaR

STaR 通过“生成 rationale → 正确则保留；错误则给正确答案再尝试 rationale → 回训 → 迭代”实现 reasoning self-bootstrapping。[3]

研究事实 | OpenThoughts

OpenThoughts 项目对 reasoning data pipeline 做了 1,000+ 受控实验；OpenThoughts3 扩到约 120 万样本，并用 QwQ-32B 作为 teacher，显示 reasoning data recipe 本身已经成为一个独立工程对象。[12]

研究事实 | s1

s1 只使用 1,000 个经 difficulty、diversity、quality 筛选的问题与 reasoning trace，就展示了“小而高价值 reasoning synthetic set + test-time control”的路线。[13]

07 | Preference / Feedback Synthesis：合成的是“相对信号”

步骤	操作	必须记录的元数据
1 Prompt selection	real / synthetic / OOD / failure prompts	source、capability、policy version
2 Candidate generation	on-policy + off-policy model pool	model_id、temperature、seed、

		timestamp
3 Aspect scoring	helpfulness / IF / truthfulness / safety / task success	judge_id、rubric_version、raw score
4 Binarization	top-vs-lower / margin threshold / disagreement routing	chosen、rejected、margin
5 Pair filtering	去低质量 chosen、无差异 pair、length/style proxy	filter version、reason

研究事实 | UltraFeedback

UltraFeedback 为约 25 万 user-assistant conversations 收集超过 100 万条 GPT-4 feedback，代表 “AI feedback as scalable alignment data” 的典型路线。[6]

研究事实 | Tulu 3

Tulu 3 将 prompt selection、model-pool response generation、GPT-4o preference annotation 串成 synthetic preference pipeline，并混合 on-policy / off-policy 与 WildChat / Persona 等来源。[10]

研究事实 | 2026 Privacy Frontier

DPPrefSyn (2026) 探索在差分隐私约束下，从私有 preference data 学习结构并合成 preference data，说明 synthetic preference 也开始承担 privacy boundary 的角色。[18]

08 | Critique、Correction 与 Negative Data：不要浪费失败样本

经典 synthetic filtering 常把错误样本全部删除，但现代训练越来越重视 “错误为何发生” 以及 “如何恢复”。这要求数据结构从单条 answer 升级为 error → diagnosis → correction。

数据形态	训练价值	注意
Bad Response + Critique	学会识别失败模式	不要把 bad response 当正向 target
Bad Step + Corrected Step	过程级纠错	需要 partial masking / step label
Rejected Pair	Preference optimization	保证 chosen 有质量下限
Counterexample	增强边界 / robustness	避免只生成 easy negative
Failed Agent Trajectory + Reflection	训练 recovery / replanning	环境 state 必须可重放

研究事实 | Agent synthetic self-reflection

STeP (2025) 合成带 reflection 与 correction 的 agent trajectories，并使用 partial masking 防止模型内化错误步骤；在 ALFWorld、WebShop、SciWorld 上报告改进。[19]

工程推论 | 工程推论：Negative Data 的价值

负样本不是 “低质量数据”，而是另一类 supervision。是否保留取决于 objective：SFT target、preference pair、critic training、process reward 的处理方式完全不同。

09 | Agent / Tool Trajectory Synthesis：从文本变成状态转移

层	普通 Chat Synthetic	Agent Synthetic
Input	Prompt	Task + Environment State
Action	文本 continuation	Tool call / file edit / browser action
Feedback	Judge score	Tool result / test / state transition
Success	response quality	task-completion state
Data unit	prompt-response	trajectory $\tau=(s_0,a_0,o_1,...,s_T)$
风险	hallucination	环境 simulator gap、long-horizon error propagation

工程推论 | 核心 Know-How

Agent Synthetic Data 最好来自可重放 environment，而不是把工具结果也让 teacher “编出来”。真实 tool execution / simulator 是 verifier，也是数据 provenance 的一部分。

10 | Generator / Teacher：强模型只是一量

维度	问题	推荐做法
Capability	Teacher 能否真正解决目标任务？	按 domain / task 路由，不迷信单一 teacher
Diversity	单 teacher 是否形成 style monoculture？	temperature / prompt style / multi-teacher / persona
Policy Relevance	teacher 输出离 student 太远吗？	难度 curriculum；混合 on/near-policy data
Calibration	teacher 自信是否可信？	外部 verifier 优先；self-score 只作辅助
Cost	高端 teacher 是否值得？	按 gap importance 与 verifier cost 做路由
License / Provenance	输出能否用于训练、来源可否审计？	记录 generator model、license、prompt、date/version

工程推论 | 一个常见误区

“更强 teacher 一定产生更好 synthetic data” 不是普适结论。Generator 的任务是制造目标 student 可学习且能验证的经验；过难、过长、过风格化的 teacher trace 可能降低有效学习率。

11 | Verifier 与过滤：Synthetic Data 的质量放大器

三角约束：防止 Synthetic Monoculture 与 Model Collapse

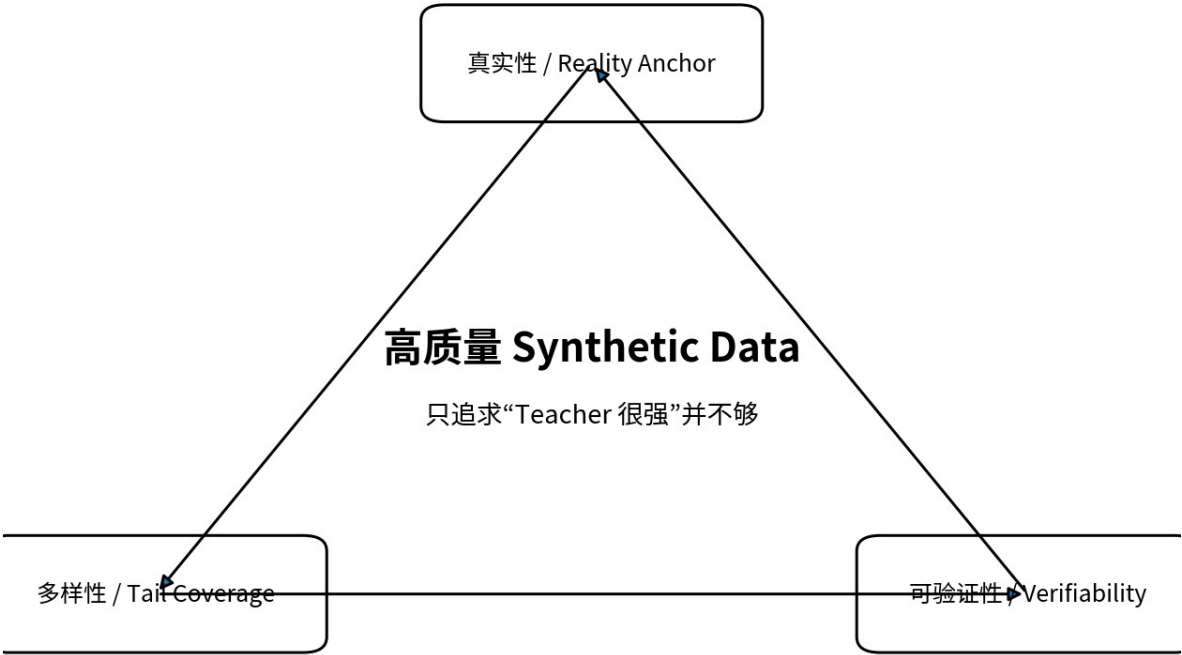


图 2 | Synthetic Data 的真实性、多样性与可验证性三角

Verifier 层级	例子	可靠性	成本	适用
Deterministic	exact match、unit test、compiler、schema	高	低-中	数学/代码/结构化输出
Symbolic/Formal	SMT / theorem prover	很高	中-高	形式化任务
Environment	任务状态是否达成	高（若环境可靠）	高	Agent / tool use
Learned RM/PRM	reward model	中	低-中	开放任务 / reasoning process
LLM Judge	rubric-guided evaluation	中	中	chat / instruction following
Human Audit	专家/标注者	高但有方差	高	gold set / disagreement / 高风险样本

工程推论 | 核心原则

能用 verifier 的部分，优先 verifier；不能可靠验证的部分，再使用 judge / reward model。Synthetic pipeline 的可扩展性最终取决于“验证是否比生成便宜”。

12 | Diversity、Difficulty 与 Learnable Frontier

Synthetic Data 最大的问题往往不是样本数不够，而是大量样本落在同一个模式附近。有效数据设计需要同时控制 semantic diversity、skill diversity、solution-path diversity 与 difficulty。

维度	监控方式	失败信号
Lexical Diversity	n-gram / MinHash / duplicate rate	大量改写而任务相同
Semantic Diversity	embedding clusters	簇高度集中
Skill Coverage	capability taxonomy coverage	目标能力缺口未覆盖
Solution Diversity	reasoning path / algorithm / tool route	所有 teacher 都用同一套路
Difficulty	student pass rate / reward entropy	全对=太易；全错=太难
Long-tail	rare task / language / constraint slices	尾部被 synthetic 生成先验抹平

工程推论 | Learnable Frontier

最值得生成的往往不是 teacher 能做出的最难题，而是当前 student “有一定概率成功、又有明显失败空间” 的区域。这与 RLVR 中 informative group 的逻辑一致。

13 | Synthetic Data Mixing：为什么必须保留 Reality Anchor

Synthetic Data 不应被视为“替代 real data”的单向迁移。更稳健的生产配方是：Real distribution 负责真实性与尾部锚定，Synthetic distribution 负责能力缺口、难度塑形和监督密度。

混合维度	不要只看	更应该看
Sample Ratio	synthetic 占多少条	Token ratio / target-token ratio
Source Ratio	real vs synthetic	每个 capability slice 的 coverage
Teacher Ratio	不同 teacher 数量	behavior/style correlation
Difficulty Ratio	easy/medium/hard 数量	student pass-rate distribution
Stage Ratio	SFT/DPO/RL 共用数据	每阶段 supervision compatibility
Time Ratio	历史数据量	fresh / stale / on-policy relevance

研究事实 | Nature Model Collapse

Shumailov 等在 Nature 2024 研究递归生成数据训练时指出，无差别地使用模型生成内容会导致原始分布尾部丢失，并称之为 model collapse；保留真实数据可缓解实验中的退化。[14]

工程推论 | 边界说明

Model-collapse 结果主要研究递归 generational training 的统计退化，不应被简化成“使用任何 synthetic data 都会 collapse”。后训练中的有验证合成、real+synthetic 混合、单轮 teacher distillation 与无约束递归训练是不同设置。

14 | Model Collapse / Synthetic Monoculture：系统性风险地图

风险	机制	症状	治理
Tail Loss	高概率模式被反复采样	长尾任务/语言消失	real anchor + tail quotas
Style Monoculture	单 teacher 输出风格主导	回答越来越模板化	multi-teacher + style normalization

Error Amplification	teacher 错误通过训练固化	同类事实/推理错误重复	external verifier + audit
Preference Echo	同模型生成 + 同模型 judge	self-confirmation	judge independence / human audit
Benchmark Echo	生成器见过 benchmark 或近似题	虚假 eval 提升	decontamination + provenance
Over-Verification Bias	只保留易验证样本	开放任务能力下降	混合 judge/human/unverified slices
Capability Narrowing	合成围绕短期 eval gap	其他能力遗忘	global regression suite + replay

工程推论 | 工程推论: Synthetic Monoculture

最危险的不一定是“模型生成数据”，而是 generator、judge、filter、eval 共享同一模型家族或同一 rubric，导致整个闭环只奖励一种表示世界的方式。

15 | Stage Router : SFT / DPO / RLVR / Distill 需要不同合成数据

Stage Router : 同一 Synthetic Pool 不应直接喂给所有训练阶段

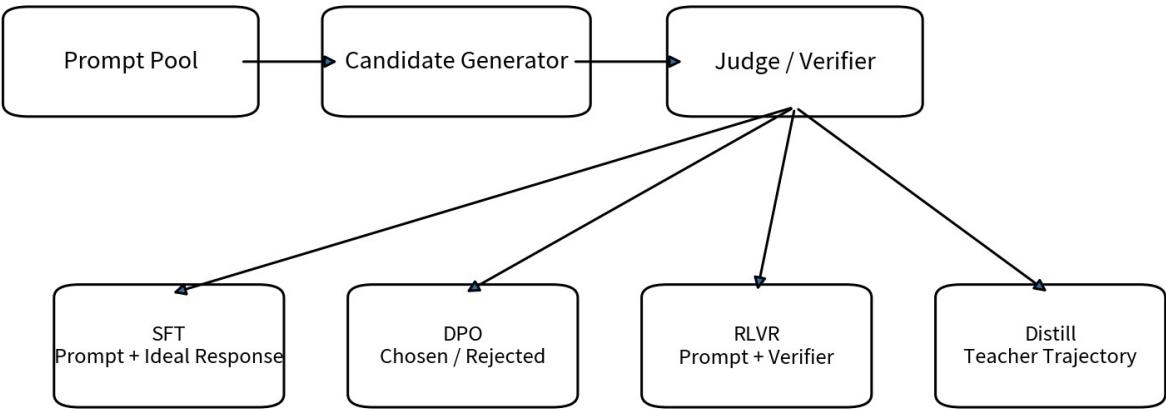


图 3 | Synthetic Stage Router

阶段	最有价值的数据	不应该直接使用
SFT	高质量 demonstration；稳定格式/行为；reasoning/tool trajectories	仅有高 reward 但过程不可模仿的 rollout
DPO	chosen/rejected 有明确相对差、chosen 质量过线、policy relevant	两个都差 / 仅长度不同 / judge 不确定 pair
RM/PRM	覆盖难例、边界例、adversarial pair/step	只含 easy separable pair
RLVR	prompt + reliable verifier + learnable frontier	没有可靠 reward 的开放任务
Distillation	强 teacher 的验证通过轨迹，适配 student 难度	远超 student 可吸收范围的冗长/复杂 trace
Agent SFT	可重放、成功或带正确 correction 的 trajectories	模拟出来但环境未执行的虚假 observation

研究事实 | DeepSeek-R1

DeepSeek-R1 的多阶段过程同时使用 cold-start data、RL、rejection sampling / SFT 与 reasoning-model distillation，说明 synthetic reasoning data 的价值依赖于训练阶段，而非单一“大数据集”。[11]

16 | Dynamic Synthetic Data Flywheel：数据应该随模型变化

Dynamic Synthetic Data Flywheel

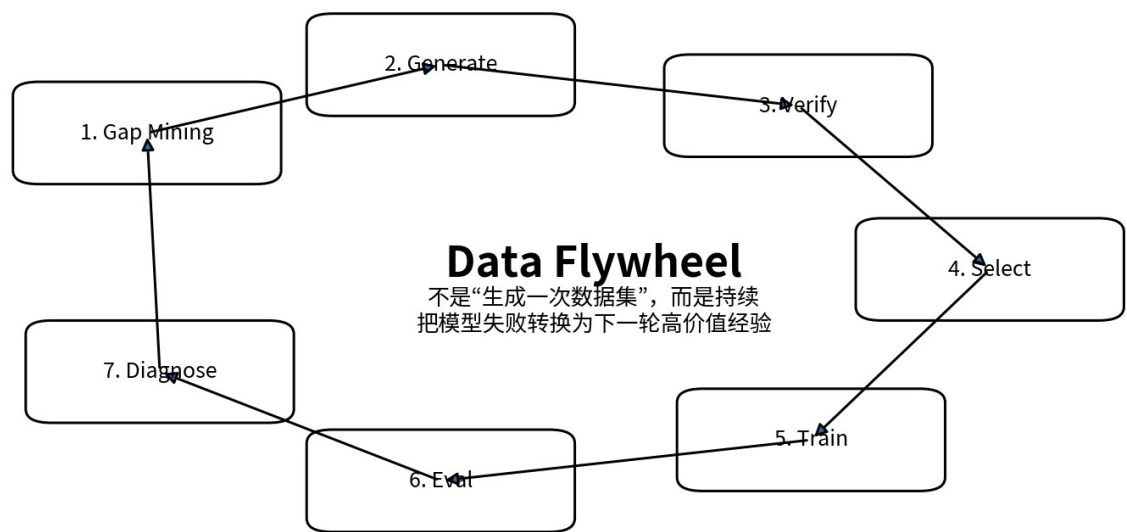


图 4 | 动态 Synthetic Data Flywheel

Eval Slice → Failure Taxonomy → Gap Priority
↓
Prompt Generator / Mutation / Persona / Source-grounded
↓
Teacher / Current Policy Rollout (N candidates)
↓
Verifier / Judge / Disagreement Gate
↓
Dedup + Difficulty + Novelty + Stage Router
↓
Train → New Policy → Re-Eval → 更新 Gap Queue

工程推论 | 动态优先级

$Priority(sample) \propto GapSeverity \times ProductImpact \times Verifiability \times Learnability \times Novelty / (GenerateCost + VerifyCost + TrainCost)$

工程推论 | 为什么旧 synthetic data 会 stale

模型进步后，过去困难样本可能变成全对的 easy data；Preference pair 也可能离当前 policy 太远。真正的数据 flywheel 必须定期重估 difficulty、policy relevance 与 pair informativeness。

17 | 成本、吞吐与基础设施：生成便宜不代表数据便宜

成本项	典型来源	优化方法
Prompt generation	LLM calls / source parsing	batching、small generator、template reuse

Response rollout	长 CoT、多候选 N	prefix cache、parallel serving、routing
Verification	tests、judge、reward model	deterministic first、cascade、early reject
Dedup / embedding	大规模 vector compute	分层 hash → embedding
Storage	raw candidates + logs + provenance	columnar storage、compression、retain policy
Human audit	disagreement / gold set	active routing 只审最有信息量样本

工程推论 | 成本函数

Synthetic Data Factory 的目标不应是“生成成本最低”，而应优化 Accepted Useful Signal / Total Cost。一个便宜但 verifier pass 率低、重复率高的 generator 可能比昂贵 teacher 更贵。

18 | 数据契约、Provenance 与污染防火墙

```
sample_id: syn_20260813_xxx
source_type: real_seed | synthetic_prompt | failure_mutation
source_id: ...
generator_model: ...
generator_version: ...
generation_config: {temperature, top_p, max_tokens, seed}
prompt_lineage: [parent_id, mutation_operator]
response_candidates: [...]
verifier: {type, version, score, evidence}
judge: {model, rubric_version, raw_scores, disagreement}
quality: {difficulty, novelty, diversity_cluster, policy_distance}
provenance: {license, dataset, url_hash, created_at}
contamination: {benchmark_match, ngram_match, semantic_match}
stage_route: [sft, dpo, rlvr, distill]
status: accepted | rejected | audit
```

必须追踪	为什么
Generator version	模型更新会改变数据分布；需要可复现
Prompt lineage	防止 mutation tree 产生大量近重复
Verifier evidence	不能只存 scalar score，要能审计为什么通过
License / source	synthetic output 仍可能继承来源与使用限制风险
Benchmark contamination	teacher 可能记忆/复现 benchmark；合成不等于无污染
Human audit decisions	用于校准 judge 与重放过滤策略

19 | 生产级 Recipes：五套可直接复用的 Synthetic Data 配方

Recipe A | General Instruction SFT Factory

- 1. Real/user seeds + persona + source-grounded prompts
- 2. Prompt validity / dedup / capability tagging
- 3. 2-4 teacher candidates per prompt
- 4. Judge rubric: correctness + IF + relevance + style

5. Hard reject hallucination / invalid format
6. Diversity-aware selection + real-anchor mix
7. SFT → behavior eval → failure prompts back to queue

Recipe B | Reasoning Distillation Factory

1. Curate hard, verifiable math/code/science prompts
2. Strong reasoning teacher generates N traces
3. Final-answer / unit-test verifier
4. Remove contaminated, duplicate, degenerate long traces
5. Difficulty bucket by student pass rate
6. SFT student on verified traces
7. Eval failures → targeted regeneration

Recipe C | Synthetic Preference Factory

1. Select real + OOD + failure prompts
2. Generate on-policy + off-policy candidates
3. Multi-aspect judge scores
4. Require chosen quality floor + minimum preference margin
5. Swap-order / disagreement checks
6. Build DPO/RM pairs; preserve raw scores
7. Re-generate when policy drifts

Recipe D | Agent Trajectory Factory

1. Task + sandbox/environment snapshot
2. Teacher agent executes real tools
3. Capture state/action/observation + tests
4. Verify task completion
5. For failures: synthesize critique + corrected trajectory
6. Mask incorrect actions where needed
7. Train agent SFT; evaluate recovery + cost + task success

Recipe E | Fully Self-Synthetic Alignment (High Risk)

1. Self-generate personas/prompts
2. Self-generate multiple responses
3. Self-evaluate / preference label
4. External audit subset + independent verifier where possible
5. Diversity quotas + real-data regression anchor
6. Train preference model / policy
7. Measure echo-bias, tail coverage, judge disagreement before scaling

工程推论 | 边界

Fully self-synthetic 可以降低人工/私有数据依赖，但风险最高：generator 与 evaluator 同源会形成闭环偏差。SAO (2025) 显示了可行性；生产使用仍应保留独立 eval 与真实分布锚点。[17]

20 | Failure Modes：Synthetic Data 最常见的 18 类失败

Failure	表象	根因	修复方向
Seed Echo	数据看似很多，任务很像	seed neighborhood 过窄	source/persona/failure 多源
Paraphrase Inflation	样本数暴涨但能力不涨	只改变语言表面	skill-level dedup
Fake Complexity	instruction 更长但不更难	evolution operator 只加约束	student pass rate / verifier
Teacher Error Amplification	同类错误反复出现	single teacher + weak verifier	ensemble / deterministic verifier
Judge Echo	synthetic preference 极度一致	generator/judge 同源	judge independence
Chosen Too Weak	DPO 后质量下降	只看 relative preference	chosen quality floor
Pair Too Easy	DPO loss 很快饱和	chosen/rejected 差太大	hard negatives / near-policy
Reasoning Hallucination	答案对但过程错	outcome-only filter	process checks / critique
Long-CoT Bloat	模型越来越啰嗦	teacher length bias	length budget / quality per token
Easy-data Saturation	训练 loss 低但无提升	样本已被 student 掌握	difficulty rebucketing
Over-Hard Data	梯度噪声/模仿失败	teacher-student gap 太大	curriculum / distill compression
Tail Erasure	稀有 slice 性能下降	高概率生成模式占据数据	real anchor / tail quota
Synthetic Monoculture	风格单一	单模型/单 rubric 全链路	multi-source / independent eval
Benchmark Leakage	离线 eval 异常高	teacher 记忆或 prompt mutation 泄漏	decontamination
Agent Simulator Gap	训练好、真实工具差	observation 是模型编的	real tool execution
Stale Synthetic Pair	DPO 后期失效	policy 已远离数据	on/near-policy regeneration
Cost Blindness	生成很多但 accepted 少	只看 token cost	useful-signal/GPU-hour KPI
No Lineage	问题无法复现	缺 provenance/version	dataset contract + registry

21 | 成熟度模型：从 L0“让模型帮忙写数据”到 L4 Adaptive Experience Factory

级别	系统形态	典型能力	关键缺口
L0 Ad-hoc	手工 prompt → LLM 批量生成	能扩数据量	无 provenance、无 verifier、无 eval loop
L1 Filtered	规则 + judge 过滤	基本质量控制	静态数据集、单 teacher
L2 Multi-source	多 prompt source + multi-teacher + dedup	覆盖/多样性管理	stage 不分、数据仍 stale
L3 Stage-aware	SFT/DPO/RLVR/Distill 路由 + verifier	训练信号匹配	反馈闭环有限
L4 Adaptive	Eval gap → dynamic generation → policy-aware difficulty → audited feedback	持续制造当前最有价值经验	系统复杂度与治理成本高

22 | 论文地图与推荐阅读顺序

顺序	论文 / 报告	为什么值得读
1	Self-Instruct [1]	理解 instruction bootstrapping 的最小闭环
2	STaR [3]	理解“生成 → 验证 → 回训”如何用于 reasoning
3	WizardLM / Evol-Instruct [4]	理解 difficulty / complexity 作为生成控制变量
4	Orca [5]	理解 teacher explanation trace 与 imitation 的差异
5	UltraFeedback [6]	理解 synthetic preference / AI feedback
6	Magpie [7]	理解从模型 prior 直接抽取 instruction 的新范式
7	Persona Hub [8]	理解 persona-driven diversity scaling
8	Nemotron-4 340B [9]	看工业级高比例 synthetic alignment pipeline
9	Tulu 3 [10]	看 SFT/DPO/RLVR 跨阶段数据 recipe
10	DeepSeek-R1 [11]	理解 reasoning synthetic / rejection / distillation 与 RL 的接口
11	OpenThoughts [12]	看 reasoning data recipe 的系统化受控实验
12	Nature Model Collapse [14]	理解递归 synthetic distribution 的统计风险
13	SAO [17] / DPPrefSyn [18]	理解 fully synthetic 与 privacy synthetic 的前沿方向

最终工程检查清单

- 是否明确合成的是 Prompt、Response、Reasoning、Preference、Trajectory 还是 Reward Label?
- 是否记录 generator / judge / verifier 的 model + version + config?
- 是否存在 independent verifier 或 human gold slice，而不是完全 self-evaluation?
- 是否测了 semantic/skill/solution diversity，而不是只做文本去重?
- 是否按 student pass rate 定义 difficulty / learnable frontier?
- 是否保留 real-user / real-data anchor 与长尾 quota?
- 是否对 SFT、DPO、RLVR、Distill 做 Stage Router?
- 是否进行 benchmark contamination / provenance 审计?
- 是否用 accepted useful signal / total cost，而不是 raw generation tokens 做 KPI?
- 是否把 Eval Failure 自动转成下一轮 synthetic generation queue?

工程推论 | 最终心智模型

Synthetic Data 的目标不是复制真实数据，也不是无限放大 teacher 输出；它的目标是在受控成本下，制造“当前模型最值得学习且可以相信”的经验。

参考资料（优先论文 / 官方技告）

- [1] Wang et al., Self-Instruct: Aligning Language Models with Self-Generated Instructions, 2022.
<https://arxiv.org/abs/2212.10560>
- [2] Honovich et al., Unnatural Instructions: Tuning Language Models with (Almost) No Human Labor, 2022.
<https://arxiv.org/abs/2212.09689>
- [3] Zelikman et al., STaR: Bootstrapping Reasoning With Reasoning, 2022.
<https://arxiv.org/abs/2203.14465>
- [4] Xu et al., WizardLM: Empowering Large Language Models to Follow Complex Instructions, 2023.
<https://arxiv.org/abs/2304.12244>
- [5] Mukherjee et al., Orca: Progressive Learning from Complex Explanation Traces of GPT-4, 2023.
<https://arxiv.org/abs/2306.02707>
- [6] Cui et al., UltraFeedback: Boosting Language Models with Scaled AI Feedback, 2023.
<https://arxiv.org/abs/2310.01377>
- [7] Xu et al., Magpie: Alignment Data Synthesis from Scratch by Prompting Aligned LLMs with Nothing, 2024.
<https://arxiv.org/abs/2406.08464>
- [8] Ge et al., Scaling Synthetic Data Creation with 1,000,000,000 Personas, 2024.
<https://arxiv.org/abs/2406.20094>
- [9] NVIDIA et al., Nemotron-4 340B Technical Report, 2024.
<https://arxiv.org/abs/2406.11704>
- [10] Lambert et al., Tülu 3: Pushing Frontiers in Open Language Model Post-Training, 2024.
<https://arxiv.org/abs/2411.15124>
- [11] DeepSeek-AI, DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, 2025.
<https://arxiv.org/abs/2501.12948>
- [12] Guha et al., OpenThoughts: Data Recipes for Reasoning Models, 2025.
<https://arxiv.org/abs/2506.04178>
- [13] Muennighoff et al., s1: Simple test-time scaling, 2025.
<https://arxiv.org/abs/2501.19393>
- [14] Shumailov et al., AI models collapse when trained on recursively generated data, Nature 2024.
<https://www.nature.com/articles/s41586-024-07566-y>
- [15] Seddik et al., How Bad is Training on Synthetic Data? A Statistical Analysis of Language Model Collapse, 2024.
<https://arxiv.org/abs/2404.05090>
- [16] Hsieh et al., Distilling Step-by-Step!, 2023.
<https://arxiv.org/abs/2305.02301>
- [17] Yin et al., Aligning Large Language Models via Fully Self-Synthetic Data, 2025.
<https://arxiv.org/abs/2510.06652>
- [18] Gao & Yang, Differentially Private Preference Data Synthesis for LLM Alignment, 2026.
<https://arxiv.org/abs/2605.30808>
- [19] Chen et al., Training LLM-Based Agents with Synthetic Self-Reflected Trajectories and Partial Masking, 2025.
<https://arxiv.org/abs/2505.20023>
- [20] Yu et al., MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models, 2023.
<https://arxiv.org/abs/2309.12284>
- [21] Zhao et al., 1.4 Million Open-Source Distilled Reasoning Dataset to Empower LLM Training, 2025.
<https://arxiv.org/abs/2503.19633>